

RESEARCH ARTICLE

To RAG or Not to RAG, That Is the Question: Effective Text-to-SQL Generation Under Ambiguity

MIA HOANG¹ AND VIJAY K. MADISETTI², (Fellow, IEEE)¹College of Computing, Georgia Institute of Technology, Atlanta, GA 30332, USA²School of Cybersecurity and Privacy, Georgia Institute of Technology, Atlanta, GA 30332, USA

Corresponding author: Vijay K. Madiseti (vkm@gatech.edu)

ABSTRACT Natural language interfaces to databases promise to democratize data access, yet current text-to-SQL systems fail to address a fundamental challenge: user questions are often ambiguous. Existing benchmarks and frameworks either ignore ambiguity entirely or require extensive training data and domain expertise during inference, limiting their practical deployment. We present two contributions to address this gap. First, we introduce Ambrosia+, a comprehensive benchmark that extends Ambrosia with 2535 unanswerable questions, creating a total of 6777 examples spanning linguistic ambiguity (scope, attachment, vagueness) and unanswerability across 16 domains and multi-table databases. Second, we propose Shakespeare-SQL, an efficient modular framework that integrates adaptive question routing, hybrid retrieval combining structural and semantic similarity, and curriculum-based Group Relative Policy Optimization (GRPO) with AI feedback for interpretation generation. Our framework achieves 72.6% full coverage on the schema-ambiguous AmbiQT benchmark using only 300 training examples—a 36.5% relative improvement over the state-of-the-art DFPL (53.2%). On Ambrosia+, Shakespeare-SQL achieves 73.0% full coverage compared to DFPL’s 30.5%, while generating 89% fewer SQL queries. On unambiguous benchmarks, our framework maintains satisfactory performance: 86.7% execution accuracy on SPIDER and 56.8% on BIRD. This work demonstrates that a strategic training methodology outweighs dataset size in ambiguity resolution, enabling production deployment without extensive and costly fine-tuning.

INDEX TERMS Text-to-SQL, natural language interfaces, query ambiguity resolution, retrieval-augmented generation, curriculum learning, group relative policy optimization, unanswerability detection, large language models, agentic AI, database management.

I. INTRODUCTION

Since its introduction in the 1970s by IBM scientists [1], SQL has been “the backbone of modern database systems,” and the same decade marked the beginning of significant research efforts focusing on translating natural language questions into SQL queries in the hope of paving the way for more intuitive user interactions with database systems and enhancing accessibility for nontechnical users [2], [3], [4], [5], [6], [7], [8]. Empowering non-technical users to query databases and quickly retrieve accurate data is becoming increasingly important as the demand for real-time insights rises and data volumes continue to grow exponentially. Delays

in extracting insights can translate directly into missed business opportunities [9].

The emergence of large language models (LLMs), marked by ChatGPT’s November 2022 launch [10], has catalyzed the development of LLM-based Text-to-SQL systems. These solutions achieve promising results on benchmarks like SPIDER and BIRD-SQL [11], [12], [13], [14], [15]. Yet despite impressive gains in benchmark performance, current systems overlook a critical obstacle: business users’ natural language questions are often ambiguous.

Ambiguity in Text-to-SQL arises when a user question might not be answerable or has multiple valid interpretations that correspond to different SQL queries. For example, “*Show me the online account enrollment trend for our portfolio X in the last 12 months*” could be interpreted as raw enrollment

The associate editor coordinating the review of this manuscript and approving it for publication was Maria Chiara Caschera¹.

numbers or enrollment percentages (total online enrollments divided by total live accounts) in the previous 12 months. Scope ambiguity, attachment ambiguity, and vagueness are common in everyday language and are now documented systematically in benchmarks such as Ambrosia [16] and AmbiQT [17].

These ambiguities pose significant risks in real-world applications, as misinterpreted SQL queries can produce syntactically valid results that do not align with the user's intentions or business requirements. This misalignment may force users to rephrase their questions multiple times or, even worse, lead to poor business decisions based on incorrect data. Existing Text-to-SQL systems typically return a single query, disregarding alternative interpretations, while evaluation protocols often limit themselves to a single ground truth, even when multiple answers are valid.

The objective of this research is to develop an efficient and effective solution that recognizes and resolves ambiguity in Text-to-SQL interactions, with performance surpassing that of state-of-the-art disambiguation frameworks. By addressing Text-to-SQL ambiguity as a core challenge rather than a peripheral issue, we aim to enhance the reliability and accessibility of natural language interfaces to databases, ultimately advancing data democratization.

II. RELATED WORK

A. BENCHMARKS FOR TEXT-TO-SQL AMBIGUITY

Early conversational Text-to-SQL datasets such as ATIS [18], ATIS-3 [19], SPaC [20], and CoSQL [21] formalized the challenges of unanswerability and underspecification within a multi-turn setting—well before large language models became mainstream. More recently, Wang et al. [22] extended the single-table WikiSQL and WTQ benchmarks to construct NoisySP, which includes:

- Answerable questions with column and value ambiguities
- Unanswerable questions referring to missing columns or values
- Questions that fall outside the executable scope of the system (e.g., requests for chart generation).

Similarly, Bhaskar et al. [17] modified the multi-table SPIDER benchmark to create AmbiQT, a benchmark designed to simulate a “dirty data” environment. In this environment, multiple columns and tables contain the same data. For example, “age” data can be stored in “age”, “years_old”, and “birthdate” columns. Each AmbiQT question allows two valid SQL interpretations that address ambiguities in columns, tables, joins, and aggregations.

Unlike earlier benchmarks, which predominantly emphasize schema ambiguity while keeping the user question clear, Ambrosia [16] focuses on linguistic ambiguity arising directly from the phrasing of the natural-language question, with three primary types: **scope**, **attachment**, and **vagueness**.

- **Scope ambiguity** arises when quantifiers such as “each,” “every,” or “all” have unclear scope. For example, “Show me the toys every cat has” can be collective (*toys*

all cats have in common) or distributive (*for each cat, list all their toys*).

- **Attachment ambiguity** occurs when modifiers can attach to multiple noun phrases. For instance, “Give me all dogs and cats who are orange”, the modifier may apply broadly to both dogs and cats or narrowly only to cats.
- **Vagueness** involves underspecified terms with multiple plausible referents. In “What technology does Rio know?”, “technology” may map to various schema elements, such as “software”, “programming language”, or both.

Interactive benchmarks, such as PRACTIQ [23], simulate dialogue turns to study ambiguous and/or unanswerable cases in a conversational pipeline. The development of these benchmarks highlights the limitations of popular single-answer and unambiguous benchmarks, such as SPIDER [24] and BIRD [25].

B. INTERNAL AMBIGUITY DETECTION AND DISAMBIGUATION

Wang et al. [22] proposed a method that detects ambiguous and unanswerable questions and provides localized explanations via a weakly supervised Detect-Then-Explain (DTE) pipeline. Huang et al. [26] demonstrated that enriching prompts with data documentation (coverage, granularity, value consistency) reduces data-level ambiguity for LLM parsers. This approach relies on high-quality documentation and yields a single SQL query as the final result. Also in 2023, Bhaskar et al. [17] introduced LogicalBeam, a novel decoding algorithm for handling ambiguous Text-to-SQL queries. Through counterfactual templates and schema infilling using beam search, LogicalBeam generates semantically different but equally valid interpretations of ambiguous queries, rather than just producing superficial variations of the same basic query.

Another significantly more complex methodology called Reliable Text-to-SQL (RTS) was proposed by Chen et al. [27]. RTS employs Branching Point Prediction (BPP) using conformal prediction techniques on LLM hidden-layer representations to provide probabilistic guarantees for error detection during SQL generation. RTS's evaluation mechanism prioritizes high precision, which requires abstention when necessary, rather than achieving high coverage. Additionally, the system will trigger human clarification when branching points are detected during generation.

On the other hand, Saporina and Lapata [28] introduced the “Disambiguate First, Parse Later” (DFPL) methodology, which focuses on identifying all possible interpretations before generating SQL queries. DFPL first generated default interpretations and their corresponding SQL queries for each question, then used a LoRA-finetuned Llama-3.1 8B Instruct model to generate any missing interpretations. Finally, Qwen-2.5-Coder-32B-Instruct translated the infilled interpretations to SQL queries.

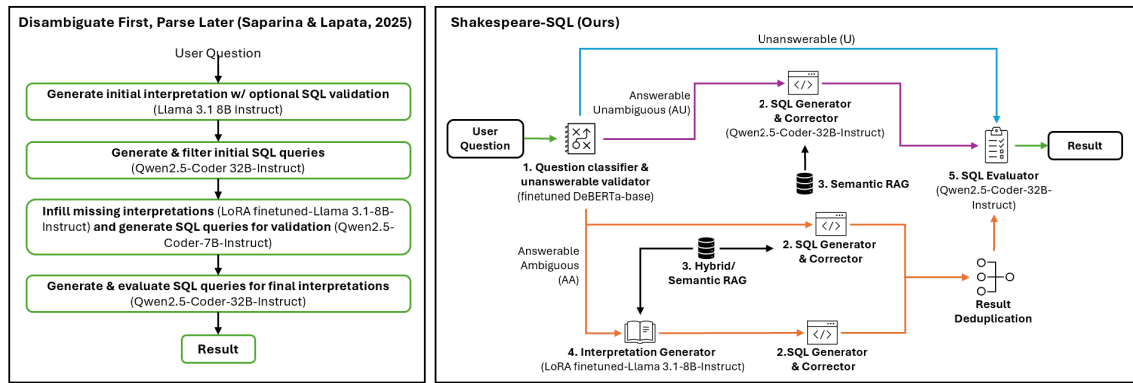


FIGURE 1. Comparison between the *Disambiguate First, Parse Later* (DFPL) pipeline (left) and our proposed **Shakespeare-SQL** framework (right) for handling ambiguity in Text-to-SQL.

C. INTERACTIVE DISAMBIGUATION

Another solution to Text-to-SQL ambiguity is to clarify user intention or seek user expertise. Qiu et al. [29] maintain a probability distribution over candidate SQLs and select clarifying questions based on their Expected Information Gain (EIG) to ask users, ultimately leading to a single final SQL query. FISQL [30], on the other hand, provides users with a rich, interactive user interface, allowing them to correct their intent and proactively repair errors. This framework assumes a higher level of domain expertise during inference, as the user must not only respond to the system's clarification requests but also initiate corrections themselves.

D. GAPS

Despite the advancement in research aiming at addressing ambiguity in Text-to-SQL generation, several gaps remain:

- 1) **Lack of comprehensive benchmarks for Text-to-SQL ambiguity:** NoisySP [22] covers ambiguity and unanswerability, but only for single-table databases. AmbiQT [17] contains only schema-ambiguous questions, while Ambrosia [16] does not include unanswerable questions, which is also a form of ambiguity. Interactive benchmarks require schema expertise, making them unsuitable for frameworks intended for non-expert users. No current benchmark covers multi-table, multi-domain scenarios with ambiguous and unanswerable queries.
- 2) **Inefficient processing pipelines:** Systems like LogicalBeam and DFPL use the same computational pathway for all questions, whether they are ambiguous, unambiguous, or unanswerable, resulting in resource inefficiency.
- 3) **Reliance costly finetuning and technical expertise during inference to resolve ambiguity:** Current approaches rely on time-consuming and expensive finetuning to achieve adequate results on AmbiQT and Ambrosia, with DFPL reaching only 30.5% full interpretation coverage without direct Text-to-SQL finetuning. Interactive disambiguation methods [23], [27], [29], [30] assume users can identify and correct

SQL or schema issues, which is unrealistic for business users without technical database knowledge. Additionally, documentation-based methods require extensive expert documentation on data and value descriptions, data coverage, data granularity, term ambiguity, and expected output [26] or complex, costly fine-tuning.

III. PROPOSED SOLUTION

Building on the work of Saparina and Lapata [16], [28], we offer two contributions to this research area:

A. AMBROSIA+

Ambrosia+ benchmark extends the original Ambrosia benchmark [16] (4,242 answerable questions across 16 domains) to include **2,535 unanswerable questions** (Table 1). We maintain DFPL's train-validation-test splits and generate three unanswerable questions for each of the 845 schemas using Claude Sonnet 4.5 and the prompt in Appendix A. Three data engineering experts validated the AI-generated unanswerable questions over two weeks and identified answerable cases, which we revised to ensure that all 2,525 questions were truly unanswerable. Finally, we used Qwen2.5-Coder-32B with the prompt in Appendix B to validate the final question set once more, ensuring the unanswerable dataset is of the highest possible quality. Ambrosia+ represents the first comprehensive benchmark for evaluating Text-to-SQL systems for non-expert users.

B. SHAKESPEARE-SQL FRAMEWORK

Shakespeare-SQL is a modular agentic framework that handles ambiguous natural-language questions over databases through a five-stage pipeline (Figure 1). The framework intelligently routes questions based on their characteristics, avoiding unnecessary computation for simple queries while providing comprehensive interpretation generation for ambiguous ones.

1) OVERVIEW OF PIPELINE ARCHITECTURE

The framework processes user questions through five interconnected components:

TABLE 1. Ambrosia+ question distribution across DFPL's train/validation/test splits by ambiguity type. Unanswerable questions were generated using the same database schemas.

Split/Type	Answerable		Unanswerable (U)	Row Total
	Unambiguous (AU)	Ambiguous (AA)	Total	
Train	2,380	1,026	2,025	5,431
Attachment	578	289	–	867
Scope	812	406	–	1,218
Vagueness	990	331	–	1,321
Validation	276	120	255	651
Attachment	80	40	–	120
Scope	88	44	–	132
Vagueness	108	36	–	144
Test	309	131	255	695
Attachment	66	33	–	99
Scope	102	51	–	153
Vagueness	141	47	–	188
Column Total	2,965	1,277	2,535	6,777

- 1) **Question Classifier & Validator:** Classifies questions as unanswerable, answerable-unambiguous, or answerable-ambiguous, with additional validation for “unanswerable” classifications
- 2) **SQL Generator & Corrector:** Produces executable SQL queries with iterative error correction
- 3) **Hybrid Retrieval-Augmented Generation (RAG):** Adaptively selects relevant examples using structural or semantic similarity to aid SQL and interpretation generations
- 4) **Interpretation Generator:** Generates multiple distinct interpretations using curriculum-trained reinforcement learning for ambiguous questions, then translates the interpretations into SQL queries
- 5) **SQL Evaluator:** Assesses query correctness through execution-based metrics

2) QUESTION CLASSIFIER & UNANSWERABILITY VALIDATOR

The framework begins by routing questions to appropriate processing paths. We employ a supervised fine-tuned DeBERTa base model (under 500 MB) that classifies questions into three categories based on the encoded natural-language question and relevant database schema:

- **Unanswerable (U):** Questions that cannot be answered given the database schema are routed directly to the evaluation agent without SQL generation. For example, “*What is the average base salary for data architects?*” is unanswerable when the schema does not contain compensation data.
- **Answerable-Unambiguous (AU):** Questions with a single clear interpretation bypass the interpretation generation step, proceeding directly to SQL generation. For example, “*Show me all employees whose home address is in zip code 07310*” has only one valid interpretation, making interpretation generation unnecessary.
- **Answerable-Ambiguous (AA):** Questions requiring disambiguation proceed through the full pipeline. For

example, “*What is the average pay for data architects?*” is ambiguous when the schema contains multiple fields representing compensation, such as “annual_salary”, “hourly_wage”, and “total_compensation”—each may require a different SQL query to retrieve the corresponding data.

Unlike prior work such as DTE [22], which requires training three separate modules (sequence labeling, grounding, and concept prediction), our framework requires only one model fine-tuned in under 50 minutes (see Section V-B for performance details).

When a question is initially classified as Unanswerable, the agent performs internal validation using the prompt in Appendix C. If validation confirms that all necessary schema elements are present and a valid SQL query can be constructed, the question is reclassified from U to AA and proceeds through the full pipeline. Otherwise, the U classification is confirmed, and no SQL generation is attempted.

3) HYBRID RETRIEVAL-AUGMENTED GENERATION (RAG)

Retrieval-Augmented Generation (RAG) [31] enhances LLM performance by retrieving relevant examples from a knowledge base before generation. Rather than using fixed examples, we implement adaptive prompting with RAG that automatically selects retrieval strategies based on question characteristics during SQL generation (for AU and AA questions) and during interpretation generation (AA questions).

a: AMBIGUITY TYPE PREDICTION

For AA questions, the system first predicts the ambiguity type using a lightweight Linear Discriminant Analysis (LDA) classifier trained on Ambrosia+’s three ambiguity categories: scope, attachment, and vague. Each question is encoded using sentence-transformers (all-MiniLM-L6-v2), then LDA projects these embeddings into a 2-dimensional space that maximizes separability among the three ambiguity types (Figure 2).

b: ADAPTIVE RETRIEVAL STRATEGIES

Based on predicted ambiguity type and confidence, the system applies different retrieval approaches:

- **Hybrid RAG:** When the LDA classifier identifies scope/attachment ambiguity with confidence ≥ 0.7 , the framework retrieves examples with the same ambiguity type from the vector database, using a weighted 70% structural similarity + 30% semantic similarity.
- **Semantic RAG:** For AU/U questions and other LDA classifier outcomes (vague ambiguity or low confidence on scope/attachment), the framework retrieves semantically similar examples using cosine similarity on sentence-transformer embeddings. We also use semantic RAG on non-Ambrosia datasets, as they lack the linguistic ambiguity that requires hybrid RAG.

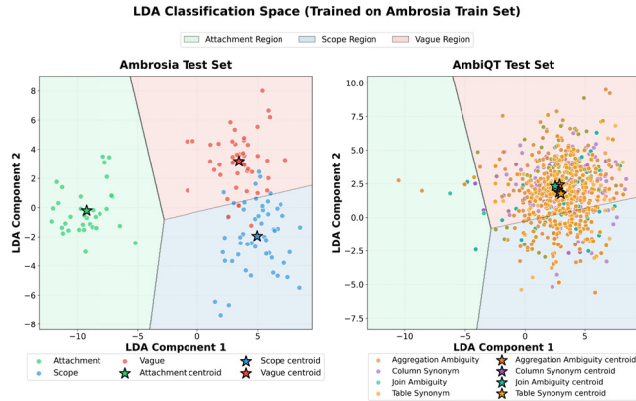


FIGURE 2. The Linear Discriminant Analysis (LDA) classifier achieves 78% accuracy on AmbiQT, in which ambiguity stems from schema inconsistencies and dirty data, and all questions map to “vague” under Ambrosia’s taxonomy. This reflects a 16% relative decrease from its in-domain accuracy (94%), indicating moderate cross-dataset robustness under a substantial distribution shift.

These parameters are determined through hyperparameter tuning with grid search using confidence thresholds $\{0.5, 0.6, 0.7, 0.8, 0.9\}$ and weight combinations $\{50/50, 60/40, 70/30, 80/20, 90/10\}$, selecting the configuration that maximizes full coverage while maintaining precision above 60%.

Our hybrid approach reflects critical differences in how ambiguity types can be resolved. Scope and attachment ambiguities follow predictable structural transformations: scope ambiguity is resolved by choosing between collective and distributive quantifier interpretations, while attachment ambiguity is resolved by determining whether modifiers apply broadly or narrowly. These structural patterns generalize across different questions, making examples with the same ambiguity *type* most useful regardless of semantic content. In contrast, vague ambiguity requires schema-specific knowledge to map underspecified terms (e.g., “technology”) to concrete schema elements (e.g., “software,” “programming languages,” or both). Since vague questions depend heavily on the specific schema and domain vocabulary, semantic similarity better identifies relevant examples.

4) SQL GENERATOR & ERROR CORRECTOR

We use Qwen2.5-Coder-32B-Instruct with RAG-retrieved examples at temperature $T = 0.0$ for deterministic behavior. Examples are retrieved using the hybrid RAG approach describe in Section III-B3 and formatted into the prompt shown in Appendix F.

Given question q , schema s , category $c \in \{AA, AU, U\}$, and RAG-based examples, the generator produces SQL queries $\{SQL_k\}_{k=1}^K$ with full autonomy over:

- **Implicit question interpretation:** Decomposing linguistic ambiguities into distinct database-level interpretations
- **Query formulation:** Selecting appropriate tables, columns, and joins
- **Error resolution:** Diagnosing and fixing SQL errors through iterative review-and-fix cycles, using the error correction prompt in Appendix G

5) INTERPRETATION GENERATOR

For AA questions that require disambiguation, we fine-tune Llama-3.1-8B-Instruct using Group Relative Policy Optimization (GRPO) [32] with curriculum learning [33] and Claude Sonnet 4.5 as the reward model. Algorithm 1 details the complete training procedure.

a: CURRICULUM LEARNING STRATEGY

We hypothesize that scope and attachment ambiguities follow structural linguistic patterns that can be learned from relatively few examples, while vagueness requires more extensive training to ground underspecified terms in a schema context. To test this hypothesis, we apply curriculum learning that progressively increases task difficulty across four training stages.

First, we apply balanced sampling to address class imbalance, selecting 100 examples per ambiguity type (300 total) from the original 1,026-example Ambrosia training set. The curriculum then proceeds through four stages: scope $\rightarrow$ attachment $\rightarrow$ vague $\rightarrow$ mixed. We train on scope and attachment examples for two epochs each, reflecting their more predictable structural patterns. Vagueness and mixed examples each receive three epochs due to their increased complexity: vagueness depends heavily on schema-specific context, and the mixed stage requires the model to integrate learning across different ambiguity types, using all 300 examples. This staged approach proves essential—initial experiments training directly on the mixed full dataset failed to converge effectively, confirming that the structural simplicity of scope and attachment ambiguities provides a foundation for learning the more context-dependent vagueness patterns.

b: GRPO OVER SUPERVISED FINE-TUNING

We choose GRPO instead of supervised fine-tuning (SFT) for three reasons: (1) SFT penalizes semantically equivalent but lexically different outputs, while GRPO rewards semantic correctness regardless of exact wording; (2) GRPO’s group-relative advantages reduce variance when ambiguity difficulty varies across examples; (3) on-policy exploration enables quality assessment through generation comparison rather than memorization. While both methods use gold interpretations I^* , GRPO uses I^* solely for reward computation $R_{\text{sem}}(s, q, y, I^*)$. This reward-based approach encourages the model to learn general patterns of ambiguity resolution rather than memorizing dataset-specific token sequences, improving cross-benchmark generalization.

c: TRAINING CONFIGURATION

We apply LoRA with rank $r = 16$, $\alpha = 32$, and dropout 0.05 following the 2:1 alpha-to-rank ratio recommended by [34]. Training uses AdamW optimizer [35] with $\beta_1 = 0.9$, $\beta_2 = 0.99$, learning rate 1×10^{-5} (standard for LoRA fine-tuning [36]), cosine learning rate schedule, and gradient clipping at $\|\nabla\|_{\text{max}} = 0.25$ following GRPO

Algorithm 1 Curriculum GRPO Training With LLM-as-Judge Reward

Input: Dataset $\mathcal{D} = \{(s_i, q_i, I_i^*, a_i)\}_{i=1}^N$ where:
 s_i = database schema/context
 q_i = natural language question
 I_i^* = gold interpretation
 $a_i \in \{\text{scope, attachment, vague}\}$ is the ambiguity type label

Input: Base policy π_{θ_0} , curriculum $\mathcal{C} = \{\text{scope, attachment, vague, mixed}\}$

Output: Optimized policy π_{θ^*}

// Initialization
 $\theta \leftarrow \text{InitializeLoRA}(\theta_0, r=16, \alpha=32, \text{dropout}=0.05)$
 $R_{\text{sem}} : \underbrace{\mathcal{S}}_{\text{schema}} \times \underbrace{\mathcal{Q}}_{\text{question}} \times \underbrace{\mathcal{Y}}_{\text{generated interp.}} \times \underbrace{\mathcal{T}^*}_{\text{gold interp.}} \rightarrow \underbrace{[0, 1]}_{\text{quality score}}$
 {Claude Sonnet 4.5 as judge}
 Partition $\mathcal{D}$ by ambiguity type: $\mathcal{D}_{\text{scope}}, \mathcal{D}_{\text{attachment}}, \mathcal{D}_{\text{vague}}$

// Curriculum Learning Loop
for stage $c \in \mathcal{C}$ **do**
 $\mathcal{D}_c \leftarrow \begin{cases} \mathcal{D}_c & \text{if } c \neq \text{mixed} \\ \mathcal{D} & \text{if } c = \text{mixed} \end{cases}$
 $E_c \leftarrow \begin{cases} 2 & \text{if } c = [\text{scope, attachment}] \\ 3 & \text{if } c = [\text{vague, mixed}] \end{cases}$
for epoch = 1 to E_c **do**
for $(s, q, I^*) \in \mathcal{D}_c$ **do**
 // On-policy sampling (group size $K = 4$)
 $\{y_k\}_{k=1}^K \sim \pi_{\theta}(\cdot | s, q)$ with temperature $T = 0.8$
 // Multi-objective reward computation
for $k = 1$ to K **do**
if $\neg \text{ValidJSON}(y_k)$ **then**
 $r_k^{\text{sem}} \leftarrow 0.25$ {Parsing penalty}
 $r_k^{\text{fmt}} \leftarrow 0.0$
else
 $r_k^{\text{fmt}} \leftarrow \text{FormatScore}(y_k) \in \{0.0, 0.01, 0.05, 0.1\}$
 $r_k^{\text{sem}} \leftarrow R_{\text{sem}}(s, q, y_k, I^*)$ {Claude evaluation}
end if
 $r_k \leftarrow r_k^{\text{sem}} + r_k^{\text{fmt}}$
end for
 // GRPO policy gradient update
 $\bar{r} \leftarrow \frac{1}{K} \sum_{k=1}^K r_k; \quad A_k \leftarrow r_k - \bar{r}$
 $\theta \leftarrow \theta - \eta \nabla_{\theta} \left[-\frac{1}{K} \sum_{k=1}^K A_k \log \pi_{\theta}(y_k | s, q) \right]$
end for
end for
 SaveCheckpoint(θ , stage= c)
end for
return π_{θ}

For each generated interpretation y_k , we compute two reward components:

- **Semantic quality reward** r_k^{sem} : Claude evaluates interpretation quality using a structured rubric (Appendix E) that prioritizes: (1) recall of ground-truth meanings (requiring semantic equivalence, not lexical overlap), (2) distinctness (penalizing duplicates and synonym rephrasing), (3) validity (schema references and SQL translatability), (4) precision (graduated penalty for excess interpretations). Scores range from [0.95, 1.0] for perfect recall with ≤ 1 extra interpretation to [0.00, 0.20] for fundamental misunderstanding.
- **Format reward** $r_k^{\text{fmt}} \in \{0.0, 0.01, 0.05, 0.1\}$: Evaluates JSON structure quality based on whether the output is valid JSON, contains a list of strings, has multiple substantial interpretations (≥ 50 characters each), and meets minimum length requirements.

When y_k cannot be parsed as valid JSON, we assign $r_k^{\text{sem}} = 0.25$ as a fixed parsing penalty. This value was selected to be higher than the minimum semantic quality score (0.20) but lower than the next tier (0.40) to distinguish parsing failure from extremely low-quality interpretations. Since total reward is $r_k = r_k^{\text{sem}} + r_k^{\text{fmt}}$, and $r_k^{\text{fmt}} = 0$ for invalid JSON, the maximum score for an invalid JSON response is 0.25.

Reward Model Selection Rationale. Claude Sonnet 4.5 was chosen as our reward model through an iterative development process. Initial experiments using GRPO with various reward models (including GPT-4o, Qwen3-Coder-30B-A3B-Instruct, and kimi-k2-thinking) yielded suboptimal results without curriculum learning. Upon discovering that curriculum-based learning substantially improved training effectiveness, we adopted Claude Sonnet 4.5 because it had performed best in our initial experiments and was already integrated into our training infrastructure. Post-hoc evaluation (Appendix I) reveals that GPT-4.1 achieves comparable performance (61.8% vs 62.0% full coverage) at significantly lower cost (\$13 vs \$34 per training run). Organizations deploying Shakespeare-SQL in production should consider GPT-4.1 as a more cost-effective alternative, particularly when training with more epochs. For organizations that favor open-source models, Qwen3-235B-A22B-Instruct-FP8 had satisfactory performance (58% full coverage) under the same training and validation settings at a higher cost (\$20) than GPT-4.1's. The key insight is that multiple proprietary LLMs and open-source models can be an effective reward model in GRPO curriculum training, as the critical factors are semantic evaluation capability and consistency, not the specific model choice.

6) SQL EVALUATOR

The SQL Evaluator is invoked after SQL generation and correction (see Figure 1). Following DFPL, we compute execution-based precision, recall, and coverage rates by comparing predicted SQL results with the gold results.

implementations [32]. Generation uses temperature $\tau = 0.8$ with vLLM for on-policy exploration.

We implement reinforcement learning with AI feedback [37] using Claude Sonnet 4.5 with temperature $T = 0$ to evaluate generated interpretations $\{y_k\}_{k=1}^K$ against ground-truth I^* . The interpretation generation prompt for Llama-3.1-8B-Instruct is in Appendix D, and the evaluation prompt for Claude is provided in Appendix E.

TABLE 2. Summary of datasets used in experiments.

Dataset	Train	Dev	Test	Domains	Ambiguity Type
Ambrosia+	5,431	651	695	16	Linguistic with AU & U cases
AmbiQT (filtered)	10,070	—	1,874	160	Schema (all AA)
BIRD Mini Dev	9,428	500	—	11	None (all AU)
SPIDER	7,000	1,034	2,147	166	None (all AU)

In production deployment without ground-truth queries, the SQL evaluator provides quality-scored explanations using semantic similarity between questions and generated interpretations, schema correctness analysis, and execution success rates (our repository [IX](#) contains full implementation details). These explanations are provided in plain English for both technical and non-technical users. An end-to-end example is provided in Appendix [H](#).

IV. EXPERIMENT SETUP

A. DATASETS

In addition to **Ambrosia+**, we evaluate Shakespeare-SQL across three additional benchmarks representing different ambiguity dimensions:

- **AmbiQT** [17] contains over 23,295 examples with schema-level ambiguity across 160 SPIDER-derived databases. Following DFPL’s preprocessing approach [28], we filtered the dataset to remove questions for which the golden SQL queries returned empty results or execution errors, yielding 10,070 train examples and 1,874 test examples.
- **BIRD DEV** [25] contains 1,534 unambiguous questions across 11 domains with realistic complexity. We selected the BIRD Mini Dev set with 500 questions for evaluation, as the test set is not public.
- **SPIDER** [24] provides 10,181 unambiguous (AU) questions across 200 databases. We used 200 samples from the training set (totaling 7,000 questions) for router augmentation and the test set (2,147 questions) for evaluation.

This combination enables evaluation of: (1) linguistic disambiguation (Ambrosia+), (2) schema disambiguation (AmbiQT), (3) routing on unambiguous queries (SPIDER, BIRD, Ambrosia+), and (4) unanswerability detection (Ambrosia+).

B. EVALUATION METRICS

To enable effective comparisons between frameworks, we adopt four evaluation metrics from DFPL [28] for answerable questions (both AA and AU):

- **Full Coverage** is the primary evaluation metric, which calculates the percentage of cases in which our framework captures all valid SQL interpretations for a given ambiguous question, where validity is determined by matching execution results against golden queries. Full coverage aligns with AmbiQT’s “BothInTopK” [17] and Ambrosia’s “All Found” metrics [16]. For instance,

consider the ambiguous question q_i : “What were the requirements for each loan?” It has three golden SQL interpretations, each retrieving a different group of columns that correspond to different meanings of “requirement”:

- Query 1: Select LoanID, Eligibility_Criteria
- Query 2: Select LoanID, Documentation_Requirements
- Query 3: Select LoanID, Eligibility_Criteria, Documentation_Requirements

To achieve full coverage for q_i , the framework must generate three queries that yield identical execution results as those of the golden queries.

- **Single Coverage** measures the proportion of cases where the framework returns at least one valid SQL interpretation, similar to AmbiQT’s “EitherInTopK” metric [17].
- **(Average) Precision and Recall** are computed at the *example level*. For each question, we calculate precision as the fraction of generated SQL interpretations that are valid, and recall as the fraction of all valid interpretations that are recovered. These per-example values are then averaged across the dataset, so the metrics reflect both the efficiency (precision) and completeness (recall) of interpretation generation.
- **Unanswerable Routing Accuracy** measures the system’s ability to correctly identify unanswerable questions and opt out of generating SQL queries for them. The capacity to detect unanswerable questions is particularly critical in high-risk domains such as banking, healthcare, and cybersecurity, where acting on incorrect data can have severe consequences. For example, in cybersecurity, a query such as “Show me all platforms running software X with a CVE¹ score above 9” presupposes that the relevant data exists to filter against. If no such values are present in the database, returning an empty result set without flagging the query as unanswerable could lead security engineers to falsely conclude that no platforms are exposed to the vulnerability, potentially leaving the door open to a threat actor.

V. RESULTS

A. MAIN RESULTS

Table 3 presents the performance of Shakespeare-SQL compared to state-of-the-art baselines across four benchmarks. Our framework demonstrates substantial improvements in addressing Text-to-SQL ambiguity while requiring significantly less training data.

1) AMBIQT PERFORMANCE

On schema-ambiguous questions, Shakespeare-SQL reaches 72.6% full coverage using just 300 Ambrosia training examples, compared to DFPL’s 53.2%, even though DFPL was fine-tuned on the complete 10,070-example AmbiQT training

¹Common Vulnerabilities and Exposures.

TABLE 3. Comparison of Shakespeare-SQL (full system with Hybrid RAG and interpretation generation) against state-of-the-art baselines across four benchmarks. Bold indicates best performance.

Test Set	Approach	Train Set Size for IG ¹	Full	Single	Recall	Precision	Total SQL Queries Generated
AmbiQT	Text-to-SQL SFT ² on AmbiQT train	10,070	63.2	82.1	72.7	58.1	–
	DFPL - finetuned on AmbiQT train	10,070	53.2	92.3	72.8	38.2	31,433
	Shakespeare-SQL (Ours) - finetuned on <u>Ambrosia</u> train	300	72.6	91.4	82.0	69.1	4,911
	Shakespeare-SQL (Ours) - No Router - finetuned on <u>Ambrosia</u> train	300	73.3	91.7	82.5	69.8	4,878
Ambrosia+	Text-to-SQL SFT on Ambrosia train	1,026	66.4	77.1	71.6	73.3	–
	DFPL - finetuned on Ambrosia train	1,026	30.5	87.8	59.7	26.3	6,831
	Shakespeare-SQL (Ours) - finetuned on Ambrosia train	300	73.0	80.7	77.4	63.7	776
	Shakespeare-SQL (Ours) - No Router - finetuned on Ambrosia train	300	78.4	87.0	82.7	51.8	1,481 ³
SPIDER Test	MiniSeek [38] ⁴	N/A	91.2	91.2	–	–	
	Shakespeare-SQL (Ours)	300	86.7	86.7	–	–	
BIRD Mini Dev	Agentar-Scale-SQL [39] ⁵	N/A	74.9	74.9	–	–	
	Shakespeare-SQL (Ours)	300	56.8	56.8	–	–	

¹ IG = Interpretation generation² All Text-to-SQL SFT performances were reported by DFPL authors in [28]³ Without the router, the agent misclassified and generated SQL queries for 50.3% of U questions⁴ #1 on SPIDER Leaderboard as of 11/2025⁵ #1 on BIRD Leaderboard as of 11/2025

Table 3: Comparison of Shakespeare-SQL (full system with Hybrid RAG and Interpretation Generation) against state-of-the-art baselines across four benchmarks. Bold indicates best performance.

set. We also beat the Text-to-SQL SFT baseline (63.2%) by 14.9 percentage points. Our recall reaches 82.0% compared to DFPL's 72.8%, and the precision difference is even more striking: 69.1% versus 38.2%, meaning we generate far fewer invalid queries.

2) AMBROSIA+ PERFORMANCE

On linguistically ambiguous questions, Shakespeare-SQL achieves 73.0% full coverage with near-perfect unanswerability detection (98.4% U routing accuracy), substantially outperforming DFPL's 30.5%. DFPL achieves higher single-coverage (87.8% vs. 80.7%), but at a cost—it generates 4.2× more queries (3,245 vs. 776), many of which are invalid. Our precision of 62.5% is significantly higher than DFPL's 26.3%. The router's impact becomes clear when we remove it: U-question routing accuracy drops from 98.4% to 49.3% as the framework returns queries for 50.7% of U questions. Without the router, Shakespeare-SQL attempts to generate SQL queries for all questions and classifies them as unanswerable only when the SQL generator returns a null value. As a result, it generates nearly twice as many queries (1,481 vs. 776). Interestingly, the no-router variant achieves higher full coverage (78.4% vs. 73.0%), suggesting that the router's main value is detecting unanswerable questions rather than improving performance on answerable ones.

3) SPIDER AND BIRD PERFORMANCE

On unambiguous benchmarks, Shakespeare-SQL achieves 86.7% execution accuracy on SPIDER, approaching MiniSeek's 91.2%. The 4.5-point gap reflects our

prioritization of ambiguity handling and efficiency. On BIRD Mini Dev, we achieve 56.8% compared to Agentar-Scale-SQL's 74.9%. The 18.1-point BIRD gap reflects a vital difference: BIRD frameworks emphasize domain-specific evidence retrieval and complex multi-hop reasoning. Top systems achieve high scores through extensive fine-tuning across BIRD's 11 domains, sophisticated data profiling, extensive evidence retrieval, and test-time scaling approaches. On the other hand, Shakespeare-SQL instead optimizes for cross-domain generalization with minimal training data (300 vs. 10,000+ examples), ambiguity resolution, and unanswerability detection—capabilities absent from BIRD-optimized systems but critical for production deployment.

For example, specialized BIRD systems excel at queries that require “oracle” knowledge to construct complex 5-table joins, but when faced with ambiguous questions like “Who made the movie?” (which could mean “wrote”, “produced”, or “directed”), these systems will likely return a single interpretation without recognizing ambiguity or unanswerability. Shakespeare-SQL represents a complementary approach that could be combined with domain-specific optimization in future work. Organizations that need both aspects could employ Shakespeare-SQL for ambiguity handling, with domain-specific systems for complex evidence retrieval and SQL-logic reasoning.

4) EFFICIENCY ANALYSIS

Shakespeare-SQL generates dramatically fewer queries than DFPL: 84% fewer on AmbiQT (4,911 vs. 31,433) and 89%

TABLE 4. Classification accuracy rates across datasets and training setups.

Dataset	U	AU	AA
(1) Trained on Ambrosia+ train set only			
SPIDER Test	–	58.6%	–
BIRD Dev	–	79.2%	–
Ambrosia+	99.6%	96.4%	87.0%
AmbiQT	–	–	16.4%
(2) Trained on Ambrosia+ train + 200 AmbiQT + 200 SPIDER			
SPIDER Test	–	99.9%	–
BIRD Dev	–	100%	–
Ambrosia+	99.2%	92.9%	93.9%
AmbiQT	–	–	100%
(3) Claude Sonnet 4.5 with Few-Shot Prompting			
Ambrosia+	96.9%	44.3%	44.3%

fewer on Ambrosia+ (776 vs. 6,831). Despite this efficiency, we achieve better coverage on both benchmarks (72.6% vs. 53.2% on AmbiQT; 73.0% vs. 30.5% on Ambrosia+), showing our approach generates higher-quality interpretations rather than just more of them.

Having established overall system performance, we now examine the question classifier that enables Shakespeare-SQL’s adaptive routing strategy.

B. CLASSIFIER PERFORMANCE

Table 4 analyzes router classification accuracy without the agent’s internal unanswerability validation, revealing why the router improves unanswerability detection while occasionally reducing coverage on answerable questions. Three configurations are evaluated: (1) Ambrosia+-only training, (2) Ambrosia+ augmented with 200 AmbiQT and 200 SPIDER examples, and (3) Claude Sonnet 4.5 with few-shot prompting.

The Ambrosia+-only model achieves strong in-distribution performance (99.6% U, 96.4% AU, 87.0% AA accuracy) but exhibits severe overfitting: it misclassifies 41.4% of SPIDER questions as unanswerable and achieves only 16.4% accuracy on AmbiQT, incorrectly routing 80.5% of schema-ambiguous questions to the unanswerable category.

Augmenting training with just 200 examples each from AmbiQT and SPIDER dramatically improves generalization, yielding near-perfect out-of-distribution accuracy (99.9% SPIDER, 100% BIRD, 100% AmbiQT) with minimal degradation on Ambrosia+ (99.2% U, 92.9% AU, 93.9% AA). The residual 92.9% AU accuracy accounts for the coverage gap between the routed (73.0%) and no-router (78.4%) variants. For reference, Claude Sonnet 4.5 with few-shot prompting achieves 96.9% on unanswerability detection but only 44.3% on AU/AA—a 50-point gap that underscores the advantage

TABLE 5. Ablation results on AmbiQT questions: full-coverage by ambiguity type.

Configuration	Column	Table	Join	Aggregation
No RAG – No IG	49.8	80.2	38.4	72.9
Semantic RAG – No IG	66.7	81.0	50.4	66.1
No RAG – IG	56.8	82.4	48.4	72.9
Semantic RAG & IG	70.1	82.4	54.4	76.8

TABLE 6. Ablation results on ambrosia+ Answerable questions: full-coverage by ambiguity type.

Configuration	Attachment	Scope	Vague	Unambig.
No RAG – No IG	3.0	0.0	0.0	70.6
Semantic RAG – No IG	33.3	52.9	0.0	80.2
Hybrid RAG – No IG	48.5	82.4	2.1	79.3
No RAG – IG	15.2	74.5	21.3	71.2
Semantic RAG & IG	42.4	82.4	14.9	80.2
Hybrid RAG & IG	57.6	94.1	17.2	79.6

of lightweight task-specific fine-tuning over general-purpose LLMs for this classification task.

These results expose a fundamental trade-off: classification errors propagate through the pipeline, reducing coverage even when individual components perform well. Nevertheless, the router’s primary value lies in unanswerability detection: perfect U routing on Ambrosia+ prevents the system from hallucinating interpretations for 255 unanswerable questions, a critical capability for production deployment. This asymmetry reflects a deliberate risk prioritization: incorrectly responding to unanswerable questions, particularly those involving prohibited or sensitive data, carries substantially greater risk than generating a single interpretation for an ambiguous question. The resulting efficiency gains are considerable: a 48% reduction in generated SQL queries (776 vs. 1,481), supporting the case that accepting marginal coverage loss in exchange for improved precision and efficiency constitutes a worthwhile engineering consideration. Future work may explore strategies to reduce routing errors, such as confidence-based thresholding, multi-stage validation, active learning from user feedback, or ensemble routing.

C. ABLATION STUDY

We systematically test each component by removing: (1) RAG-based prompting, (2) the fine-tuned interpretation generation model, and (3) hybrid RAG’s structural component. Tables 5 and 6 show how full coverage changes across different ambiguity types.

Semantic RAG substantially improves column ambiguity resolution on AmbiQT, increasing accuracy from 49.8% to 66.7% by enabling the SQL generator to identify alternative schema elements. The effect is more pronounced on Ambrosia+, where scope ambiguity resolution improves from

TABLE 7. Full coverage performance comparison of different LoRA adapters in the same zero-shot setting with routing enabled.

LoRA Adapter	Train Set	Train Set Size	Attachment	Scope	Vague	Overall
DFPL	AmbiQT	10,070	0.0	15.7	17.2	53.4
DFPL	Ambrosia	1,026	3.0	64.7	29.8	60.7
DFPL	AmbiQT & Ambrosia	11,096	3.0	66.7	31.9	61.4
Ours	Ambrosia	300	15.2	74.5	21.3	62.0

0.0% to 52.9% and attachment ambiguity from 3.0% to 33.3%. These results indicate that without RAG, Qwen2.5-Coder-32B is insufficient for generating multiple interpretations from the schema alone.

The interpretation-generation (IG) component yields the greatest gains for ambiguity types that require explicit enumeration. On AmbiQT, it produces modest improvements for column (+7.0% compared to No RAG - No IG) and join (+10.0%) ambiguity, while contributing negligibly to table ambiguity resolution. On Ambrosia+, it proves critical for vague ambiguity, improving resolution from 0.0% to 21.3%, where the system must link underspecified terms to multiple schema elements.

Hybrid RAG outperforms semantic-only RAG on Ambrosia+ for both attachment (48.5.6% vs. 33.3% in the setting without IG) and scope ambiguity (82.4% vs. 52.9%), confirming that structurally similar retrieved examples are better suited to these systematic ambiguity patterns.

Architectural Comparison with DFPL. RAG+SQL and DFPL differ fundamentally in how they order their generation pipeline. DFPL follows a four-stage process in Figure 1, which introduces two failure modes. First, interpretation-to-SQL translation errors compound across stages: even when Llama produces correct interpretations, the SQL generator may misparse them. For instance, given the attachment-ambiguous question "Show me lobbies and terraces that can accommodate 500 guests", DFPL's finetuned Llama correctly generates one interpretation in which the 500-guest constraint applies only to terraces, yet Qwen incorrectly applies it to both. Second, the pipeline relies on Llama to meta-reason about the completeness of interpretations—a task for which the LoRA-finetuned model is not well-suited.

RAG+SQL sidesteps both issues by restructuring the pipeline: Qwen first generates initial SQL queries by analyzing in-context RAG examples, Llama then produces interpretations conditioned on the ambiguous question, and Qwen finally generates SQL for these interpretations before deduplication. This ordering allows Qwen to make an initial attempt at SQL generation and removes the burden of completeness reasoning from Llama entirely.

D. LORA ADAPTER COMPARISON

Table 7 compares interpretation generation adapters under identical conditions, isolating the effects of training

TABLE 8. Execution results for both generated SQL queries.

Query 1 Results		Query 2 Results	
Municipality	Foreign Born	Municipality	Naturalized
New York City	5,000	New York City	3,000
Los Angeles	3,500	Los Angeles	2,500
Chicago	2,500	Chicago	2,000

methodology and data selection. Our GRPO-trained adapter achieves 62.0% full coverage on Ambrosia+ using only 300 training examples and 2–3 epochs per stage, outperforming DFPL adapters trained on substantially larger datasets over 15 epochs each.

Our adapter utilizes 97% less training data than the strongest DFPL baseline (300 vs. 11,096 examples) while attaining higher overall performance, demonstrating that training methodology is a more significant factor than dataset scale. The proposed curriculum-based GRPO approach requires only 2–3 epochs per stage on 100 examples (except for the mixed stage, which uses all 300 examples), compared to 15 epochs for DFPL, reducing training time by approximately 75%. This efficiency can be attributed to three factors: curriculum learning that progressively stages examples by difficulty, reward-based optimization via GRPO that targets semantic correctness, and balanced sampling across ambiguity types.

Furthermore, the Shakespeare-SQL adapter substantially outperforms DFPL on scope (74.5% vs. 66.7%) and attachment ambiguity (15.2% vs. 3.0%), which we attribute to curriculum learning's capacity to capture systematic structural patterns. DFPL achieves higher performance on vague ambiguity (31.9% vs. 21.3%), likely owing to the greater coverage of term-grounding examples afforded by its larger training corpus. Nevertheless, the gains in scope and attachment ambiguity resolution more than offset the decline in vague ambiguity performance.

The $37\times$ reduction in LoRA training data (11,096 $\rightarrow$ 300 examples), accompanied by superior performance, suggests that Text-to-SQL research may place disproportionate emphasis on dataset size rather than on training methodology. This finding carries meaningful practical implications: state-of-the-art ambiguity resolution does not require exhaustive data collection but rather carefully curated training samples, principled curriculum design, and reward-based optimization that prioritize semantic correctness.

VI. LIMITATIONS

While Shakespeare-SQL demonstrates substantial improvements in Text-to-SQL ambiguity resolution, several limitations merit discussion.

A. DOMAIN-SPECIFIC QUERY COMPLEXITY

Shakespeare-SQL achieves 56.8% execution accuracy on BIRD Mini Dev, compared to 74.9% reported by specialized systems. This gap reflects a deliberate design trade-off that prioritizes ambiguity resolution over domain-specific

TABLE 9. Full coverage performance comparison of LoRA adapters trained by different reward models in the same zero-shot setting with routing enabled.

Reward Model	Attach.	Scope	Vague	Overall	Token Cost/ Training Run	GPU Rental Cost (2.5 hrs/Training Run)
claude-sonnet-4.5-20250929	15.2	74.5	21.3	62.0	\$30	\$4 (1 x RTX PRO 6000)
gpt-4.1-2025-04-14	18.2	70.6	23.4	61.8	\$9	\$4 (1 x RTX PRO 6000)
gpt-5.2-2025-12-11	6.1	49.0	12.8	57.3	\$27	\$4 (1 x RTX PRO 6000)
Qwen3-235B-A22B-Instruct-2507-FP8	9.1	49.0	12.8	58.0	–	\$20 (5 x RTX PRO 6000)

optimization. Deployments requiring peak performance on complex, unambiguous queries may necessitate supplementing Shakespeare-SQL with domain-specific fine-tuning.

B. VAGUE AMBIGUITY COVERAGE

The proposed system achieves 23.4% full coverage on vague ambiguity, compared to 31.9% for DFPL. This disparity stems from our balanced sampling strategy (100 examples per ambiguity type), which prioritizes training efficiency over maximizing performance on any single ambiguity category. Domains characterized by a high prevalence of vague queries may require additional training data to improve term disambiguation coverage.

C. ROUTER ERROR PROPAGATION

Although the ambiguity classifier achieves 93.9% accuracy, misclassifications propagate through the downstream pipeline. Ablation results show that removing the router improves full coverage on answerable questions (78.4% vs. 73.0%), indicating that classification errors impose a measurable performance penalty despite the efficiency benefits of routing. This trade-off is considered acceptable in applications where unanswerability detection is a critical system requirement.

D. LANGUAGE AND DIALECT GENERALIZABILITY

Shakespeare-SQL is evaluated exclusively on English-language questions against SQLite databases. Generalization to other natural languages and SQL dialects, including PostgreSQL, MySQL, and Oracle, has not been validated. Furthermore, domain-specific SQL constructs such as window functions, common table expressions (CTEs), and JSON operators are underrepresented in the training data, potentially limiting performance in specialized enterprise contexts.

E. STATIC SCHEMA ASSUMPTION

The framework assumes schema stability throughout inference. Production environments subject to schema evolution would require mechanisms to detect structural changes and update retrieval indices accordingly. Dynamic schema elements,

such as views and stored procedures, which are common in enterprise deployments, are not currently accounted for in the framework.

VII. FUTURE WORK

Several promising directions emerge from this research:

A. MULTILINGUAL AMBIGUITY RESOLUTION

Extending Shakespeare-SQL to non-English languages presents unique challenges, as ambiguity types manifest differently across languages. Scope ambiguity in quantifier-final languages, such as Japanese and Korean, differs structurally from English. Developing language-specific ambiguity taxonomies and multilingual interpretation-generation models would broaden their applicability to global enterprises.

B. INTERACTIVE DISAMBIGUATION

While our system returns multiple SQL interpretations, a more user-friendly approach would be to provide natural-language clarification. Given “Show me math courses and the resources available for them”, the system could ask: “What do you mean by resources? We have: (1) textbook, (2) software, (3) office hours, (4) teaching assistants.” This preserves automatic generation while making disambiguation accessible to non-technical users who understand their domain intent but lack SQL expertise.

C. CONTINUAL LEARNING FROM USER FEEDBACK

Production deployments generate feedback through implicit signals (query reformulations, result selections) and explicit corrections. Developing online learning mechanisms that update interpretation models and RAG indices based on user interactions could enable continuous improvement, but this would require careful privacy considerations and feedback quality assessment.

D. CONVERSATIONAL CONTEXT INTEGRATION

Current benchmarks evaluate single-turn interactions, but real usage involves multi-turn conversations with accumulating context. Extending Shakespeare-SQL to maintain conversation history, resolve anaphoric references, and handle follow-ups would improve practical utility by specifying when to inherit versus reset context.

E. ENHANCED VAGUE AMBIGUITY COVERAGE

Our balanced sampling (100 examples per type) prioritized cross-type generalization and training efficiency. However, vague ambiguity’s lower coverage (23.4% vs. DFPL’s 31.9%) suggests this type benefits more from additional training data. Future work should explore whether increasing the number of vague-ambiguity examples improves coverage without requiring proportional increases for other types.

VIII. CONCLUSION

Ambiguity remains a fundamental obstacle to democratizing database access through natural language interfaces. We

address this challenge through two contributions: Ambrosia+, the first benchmark that combines linguistic ambiguity and unanswerability across multi-table domains, and Shakespeare-SQL, a modular framework that achieves state-of-the-art disambiguation with 97% less training data than prior work.

Shakespeare-SQL integrates three key innovations: adaptive routing via a lightweight question classifier for efficient resource allocation, hybrid retrieval combining structural and semantic similarity for improved prompt construction, and curriculum-based GRPO training with AI feedback for sample-efficient interpretation generation. Together, these components achieve 72.6% full coverage on AmbiQT and 73.0% on Ambrosia+ with perfect unanswerability detection, substantially outperforming prior state-of-the-art systems.

This work yields two broader insights for the design of Text-to-SQL systems. First, training methodology is a more decisive factor than dataset scale—our curriculum-based approach with 300 curated examples outperforms standard fine-tuning on 11,096 examples. Second, ambiguity-aware routing reduces computational overhead and improves precision by concentrating resources where they are most consequential.

The practical significance of these contributions extends beyond benchmark evaluation. In high-stakes domains such as banking and cybersecurity, reliable unanswerability detection and the generation of multiple valid interpretations transform Text-to-SQL from a research artifact into a deployable production capability. Perfect unanswerability detection eliminates silent failures that could precipitate regulatory violations or security vulnerabilities, while multiple SQL interpretations preserve user agency in query intent resolution.

Addressing the remaining limitations, including domain-specific query complexity, multilingual generalization, and real-world deployment validation, will further advance the utility of natural language database interfaces. By treating ambiguity and interpretive trust as first-class design concerns, this work takes a meaningful step toward data systems that reliably serve both technical and non-technical users.

IX. CODE AND DATA AVAILABILITY

All files, including the Ambrosia+ dataset are available at: <https://github.com/miahoang/shakespeare-sql>

APPENDIX A PROMPT TO GENERATE UNANSWERABLE QUESTIONS USING CLAUDE SONNET 4.5

You are helping to generate unanswerable questions for a Text-to-SQL benchmark dataset.

Domain: {*domain*}

Database Schema: {*schema*}

Example questions from this domain:

textitexample_questions

Task: Generate 3 unanswerable questions that CANNOT be answered given the database schema above.

Unanswerable questions should be realistic and seem plausible at first glance, but require information that is NOT present in the database schema. For example:

- Asking for columns that don't exist (e.g., "What is the average salary?" when there's no salary column)
- Asking about tables that don't exist (e.g., "List all products" when there's no products table)
- Asking for relationships that cannot be established with the available tables
- Asking for temporal information when timestamps aren't available
- Asking for aggregations or filters on non-existent columns or tables

Requirements:

- 1) Each question should be natural and realistic for this domain
- 2) Each question should be clearly unanswerable given ONLY the schema provided
- 3) Questions should vary in type and complexity
- 4) Questions should be similar in style to the example questions
- 5) Avoid repetitive patterns

Return ONLY a JSON array of 3 question strings, no additional explanation.

Format: ["question 1", "question 2", ...]

APPENDIX B PROMPT TO VALIDATE UNANSWERABLE QUESTIONS GENERATED BY CLAUDE SONNET 4.5

Given the database schema and the natural-language question, determine whether the question can be answered and, if so, generate SQL queries.

Database Schema: {*db_dump*}

Question: {*question*}

Task:

Carefully analyze if this question can be answered with the given schema by checking:

- 1) **Required Tables:** Do all necessary tables exist in the schema?
- 2) **Required Columns:** Do the required columns exist in those tables?
- 3) **Joins (if needed):** If data from multiple tables is required, can they be joined?
 - Check for foreign key relationships (e.g., table1.id referenced by table2.table1_id)
 - Check for common columns that enable joins
 - Verify join columns have compatible types
- 4) **Data Availability:** Does the schema contain the information needed to answer the question?

If ANY of these checks fail, the question is UNANSWERABLE with this schema.

Output Format (JSON only): {{ "can_answer": true or false, "sql": "SELECT ...FROM ...WHERE ..." or null, "missing_elements": {{ "tables": ["list of missing tables"], "columns": ["list of missing columns"], "joins":

“explanation if tables cannot be joined” }} or null, “reasoning”: “Detailed explanation of why answerable or not” }}

Important: - If can_answer is true, you MUST provide at least one valid SQL - If can_answer is false, you MUST explain what’s missing - Be thorough in checking schema compatibility

Respond with ONLY the JSON object, no other text.

APPENDIX C

PROMPT TO VALIDATE UNANSWERABLE CLASSIFICATION DURING INFERENCE

Validate whether the following question is truly UNANSWERABLE given the database schema.

Question: {question}

Database Schema: {schema}

Your task:

1. Carefully analyze whether this question can be answered with the given database
2. Check if ALL required information exists in the schema (tables, columns, data)
3. A question is UNANSWERABLE if:

- It asks for data/columns that don’t exist in the schema (e.g., “box office revenue” when only “budget” exists)
- It asks about entities not in the database (e.g., “Action” genre when only “Horror” and “Thriller” exist)
- The required relationships between tables don’t exist
- The question is logically impossible to answer with the available data

4. A question is ANSWERABLE if:

- All required columns exist in the schema
- All required data values exist or could exist in the tables
- The question can be translated to a valid SQL query that would return meaningful results

Provide your analysis in the following format:

VALIDATION: [CONFIRMED_U/SHOULD_RECLASSIFY]

CONFIDENCE: [0.0-1.0]

REASONING: [Detailed explanation of why this question is or isn’t answerable, specifically listing what’s missing or available]

APPENDIX D

PROMPTS FOR INTERPRETATION GENERATION DURING CURRICULUM GRPO TRAINING

A. SYSTEM PROMPT

You are an expert database question analyst specializing in disambiguating ambiguous natural language queries.

Your task: Given a database schema and an ambiguous question, identify and generate ALL semantically distinct interpretations of the question.

The question can contain different types of linguistic ambiguity:

1. SCOPE AMBIGUITY: Quantifier scope (“each”, “every”, “all”) is unclear
 - Example: “Show me the toys every cat has.”

- Collective interpretation (property shared by ALL entities together): “List the toys that all cats have in common.”
- Distributive interpretation (property of EACH entity individually): “For each cat, list their toys.”

2. ATTACHMENT AMBIGUITY: “X and Y [modifier]” - modifier attachment is unclear

- Example: “Give me dogs and cats who are orange. Show them in one table.”
- Broad scope (modifier applies to BOTH X and Y): “Select all dogs who are orange AND cats who are orange.”
- Narrow scope (modifier applies ONLY to Y): “Select ALL dogs (no color filter) AND cats who are orange.”

3. VAGUENESS: Underspecified terms have multiple possible referents

- Example: “What technology does Rio know?”
- Interpretation 1: “What software does Rio know?”
- Interpretation 2: “What programming languages does Rio know?”
- Interpretation 3: “What software and programming language does Rio know?”

Guidelines for generating interpretations:

- Interpretations must be in natural language (not SQL)
- Follow the examples above for each ambiguity type closely (how many interpretations you should generate and how you should phrase them)
- Each interpretation must represent a DIFFERENT possible meaning or intent
- Interpretations should differ in WHAT data is being requested, not just HOW it’s phrased (if they lead to the same SQL query/result, they are NOT distinct)
- Ground each interpretation in the actual schema (tables, columns, relationships)
- Make each interpretation specific and executable (could be translated to SQL)
- Avoid synonym variations - focus on semantic differences

Output format: Return ONLY a JSON array of strings. No explanations, no prose, no markdown code blocks.

Example: [”interpretation 1”, “interpretation 2”, “interpretation 3”]

B. USER PROMPT

Given:

Database Schema: {schema}

Ambiguous Question: {question}

Task: List every distinct way the question could be understood.

Each interpretation should:

- 1) Be grounded in the provided schema
- 2) Represent a different valid meaning (not synonym variations)
- 3) Be specific enough to translate into SQL
- 4) Provide the CORRECT information to the question asked

Return ONLY a valid JSON array of strings. No other text, no explanations, no markdown formatting.

APPENDIX E

PROMPT FOR CLAUDE SONNET 4.5 TO EVALUATE INTERPRETATIONS DURING CURRICULUM GRPO TRAINING

A. SYSTEM PROMPT

You are an expert evaluator of database question interpretations. Your task is to score how well generated interpretations match ground-truth (GT) interpretations.

INPUT:

- 1) Database schema
- 2) Ambiguous question
- 3) Generated interpretations (candidate set)
- 4) Ground-truth interpretations (GT set with N items)

UNDERSTANDING GT INTERPRETATIONS:

The GT interpretations represent semantically distinct meanings due to linguistic ambiguity:

1. **SCOPE AMBIGUITY:** Quantifier scope (“each”, “every”, “all”) is unclear

- Example: “Show me the toys every cat has.”
- Collective interpretation (property shared by ALL entities together): “List the toys that all cats have in common.”
- Distributive interpretation (property of EACH entity individually): “For each cat, list their toys.”

2. **ATTACHMENT AMBIGUITY:** “X and Y [modifier]” - modifier attachment is unclear

- Example: “Give me dogs and cats who are orange. Show them in one table.”
- Broad scope (modifier applies to BOTH X and Y): “Select all dogs who are orange AND cats who are orange.”
- Narrow scope (modifier applies ONLY to Y): “Select ALL dogs (no color filter) AND cats who are orange.”

3. **VAGUENESS:** Underspecified terms have multiple possible referents

- Example 1: “What technology does Rio know?”
 - Interpretation 1: “What software does Rio know?”
 - Interpretation 2: “What programming languages does Rio know?”
 - Interpretation 3: “What software and programming language does Rio know?”
- Example 2: “What was the focus of the conference?”
 - Interpretation 1: “What was the topic of the conference?”
 - Interpretation 2: “What was the theme track of the conference?”
 - Interpretation 3: “What was the theme track and topic of the conference?”

IMPORTANT:

- The GT interpretations specify WHICH schema element(s) the vague term maps to.

- Generated interpretations MUST match these specific schema mappings, not just paraphrase the original vague term.

EVALUATION CRITERIA:

VALIDITY CHECK (for each generated interpretation):

- Schema-grounded: references actual tables/columns from the schema
- Executable: specific enough to translate to SQL unambiguously
- Semantically distinct: would produce different SQL query or result set from other interpretations

MATCHING CHECK:

- A generated interpretation MATCHES a GT interpretation if both would lead to the same SQL query
- Paraphrases/synonyms that yield identical SQL are the SAME interpretation (not distinct)

SCORING FRAMEWORK:

Note: N = number of GT interpretations

Note: Recall is MOST important. Precision matters less - a few extras are acceptable.

Step 1 - **RECALL:** How many of the N GT meanings were captured?

- Count exact matches (interpretations that capture the EXACT same semantic meaning as GT)
- This is the PRIMARY scoring factor

Step 2 - **DISTINCTNESS:** Are generated interpretations truly distinct from each other?

- Duplicates: multiple generated interpretations that map to the same SQL
- This is a MODERATE penalty

Step 3 - **VALIDITY:** Are interpretations schema-grounded and executable?

- Invalid: references non-existent tables/columns or too vague to translate to SQL
- This is a MODERATE penalty

Step 4 - **PRECISION:** How many extras beyond GT?

- One extra is acceptable - MINOR penalty
- A few extras (2-3) suggest lack of focus - MODERATE penalty
- Extreme extras (4+) suggest poor understanding - HEAVY penalty

SCORING RUBRIC (0.0 to 1.0):

0.95-1.00: Perfect

- Captured all N GT meanings
- All valid and distinct
- 0-1 extra interpretations
- Example: GT=2, Generated=2-3 (all valid, all distinct, all GT captured)

0.80-0.90: Excellent

- Captured all N GT meanings (perfect recall!)
- Minor issues: 1-2 duplicates OR extras
- Example: GT=2, Generated=3-4 but captured both GT meanings

0.65-0.75: Good

- Captured N-1 GT meanings (missing just one)
- 1-2 duplicates or extras, but mostly valid
- Example: GT=3, Generated=4, captured 2 GT meanings

0.45-0.60: Fair

- Captured roughly half of GT meanings
- For N=2: captured 1
- For N=3: captured 1-2
- May have 2-3 duplicates, extras, or validity issues

0.25-0.40: Poor

- Captured few GT meanings (0-1 when N=2-3)
- Multiple issues: duplicates, invalids, or extreme over-generation (7+)

0.00-0.20: Very Poor

- Captured 0 GT meanings OR
- All/mostly invalid interpretations OR
- Fundamental misunderstanding

CRITICAL OUTPUT FORMAT:

- **RETURN ONLY A SINGLE NUMBER**
- No explanation, no text, no additional characters - just the number.

Examples of valid output: 0.85 or 1.0 or 0.42

B. USER PROMPT

Database Schema: *{schema}*

Ambiguous Question: *{question}*

Ground Truth Interpretations:

{ground_truth_interpretations}

Generated Interpretations:

{generated_interpretations}

Respond with ONLY a single number between 0.0 and 1.0.

APPENDIX F

PROMPT TO GENERATE SQL QUERY DURING INFERENCE

A. SYSTEM PROMPT

You are an expert SQL query generator for ambiguous questions.

Your task: Write SQL queries based on the provided question. Questions can be ambiguous, meaning they can be interpreted in different ways. In such cases, write all possible SQL queries corresponding to different interpretations.

Return your response in JSON format:

```
{
  "sql_queries": ["SELECT ...", "SELECT ..."]
}
```

Requirements:

- **DO NOT SELECT EXTRA COLUMNS** beyond those requested in the question
- Do not include any explanations or interpretations
- Review the actual database schema to identify all possible interpretations (e.g., column synonyms, table synonyms)
- Return valid JSON only - start with { and end with }
- In JSON strings, escape backslashes as \\ (double backslash)

- In JSON strings, escape double quotes as \"
- Use double quotes for SQL string literals (e.g., WHERE name = "John")
- No markdown code blocks, no extra explanations
- Make sure your JSON response is valid before returning it

B. USER PROMPT

Some example questions and their SQL queries are provided below based on similar problems:

{examples_text}

Given the following SQLite database schema: *{schema}*

Answer the following: *{question}*

Return JSON format:

```
{
  "sql_queries": ["query1", "query2"]
}
```

APPENDIX G

PROMPT TO CORRECT SQL QUERY DURING INFERENCE

A. SYSTEM PROMPT

You are an expert SQL validator for SQLite.

You will be given a question, a database schema, and a list of SQL queries that were generated to answer the question.

Your task: Decide whether the queries correctly answer the question and whether they contain any syntax errors, then fix any issues you have identified.

Rules:

- Review the question and database schema to understand the intent
- Identify SQLite syntax errors (missing commas, incorrect keywords, etc.)
- Identify selection errors (wrong/missing/extra columns, incorrect table references)
- Check whether the query sorts, filters, and/or aggregates data as intended by the question
- If there are any errors, correct them to ensure the query runs successfully and returns the data the question is asking for

CRITICAL JSON FORMATTING RULES:

- Return valid JSON only - start with { and end with }
- In JSON strings, escape backslashes as \\ (double backslash)
- In JSON strings, escape double quotes as \"
- Use double quotes for SQL string literals (e.g., WHERE name = "John")
- No markdown code blocks, no extra explanations

Return format:

```
{
  "corrected_queries": ["SELECT ...", "SELECT ..."]
}
```

B. USER PROMPT

Some example questions and their SQL queries are provided below based on similar problems: *{examples_text}*

Given the following SQLite database schema: {*schema*}
 Question: {*question*}
 Interpretations and queries to review: {*queries_text*}
 The following execution errors occurred with the queries above: {*errors_text*}

Review the question, interpretations and queries thoroughly, then fix any issues you can identify.

IMPORTANT: When reviewing queries and removing duplicates, only remove **EXACT** duplicates (identical SQL text).

Return JSON only:

```
{
  "corrected_queries": ["corrected_query1", "corrected_query2"]
}
```

APPENDIX H

END-TO-END EXAMPLE: PROCESSING AN AMBIGUOUS QUESTION

This section illustrates how Shakespeare-SQL processes a single ambiguous question through all pipeline stages, from classification to final SQL generation.

A. INPUT QUESTION AND SCHEMA

Question: "Show me cities and immigrants where more than 60,000 people live"

Database Schema:

```
CREATE TABLE Population (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  Total_Population INT,
  Foreign_born_Population INT,
  Naturalized_Citizens INT,
  Municipality TEXT
);
```

Sample Data:

Municipality	Total Pop.	Foreign Born	Naturalized
New York City	100,000	5,000	3,000
Los Angeles	80,000	3,500	2,500
Chicago	70,000	2,500	2,000
Houston	60,000	2,000	1,500
Philadelphia	50,000	1,500	1,000

B. STAGE 1: QUESTION CLASSIFICATION

The fine-tuned DeBERTa classifier analyzes the question and schema:

- **Classification Result:** AA (Answerable-Ambiguous)
- **Routing Decision:** Proceed through the full pipeline with interpretation generation

C. STAGE 2: HYBRID RAG RETRIEVAL AND INITIAL SQL GENERATION

Ambiguity Classification: Vague (term "immigrants" is underspecified)

RAG Strategy: Semantic retrieval (30% weight on semantic similarity using sentence-transformer embeddings, 70% weight on structural similarity via LDA projection). Since this is vague ambiguity, semantic similarity dominates the retrieval.

Retrieved Examples (5 total):

Example 1: "Show all countries that have a population less than 100,000,000 and their languages."

Valid SQL Queries (3):

SQL 1:

```
SELECT Name, Official_Language
FROM Countries
WHERE Population < 100000000
```

SQL 2:

```
SELECT Name, Spoken_Language
FROM Countries
WHERE Population < 100000000
```

SQL 3:

```
SELECT Name,
Official_Language,
Spoken_Language
FROM Countries
WHERE Population < 100000000
```

Example 2: "Which country in North America had the most immigrants?"

Valid SQL Queries (2):

SQL 1:

```
SELECT C.countryName
FROM Country C
INNER JOIN ImmigrationStats I
ON C.id = I.countryId
WHERE C.continent = 'North America'
ORDER BY I.numberOfImmigrants DESC
LIMIT 1
```

SQL 2:

```
SELECT C.countryName
FROM Country C
INNER JOIN ImmigrationStats I
ON C.id = I.countryId
WHERE C.continent = 'North America'
ORDER BY I.immigrantPercentage DESC
LIMIT 1
```

Example 3: "Show the country with the smallest population and its language."

Valid SQL Queries (3):

SQL 1:

```
SELECT Name, Official_Language
FROM Countries
ORDER BY Population ASC
LIMIT 1
```

SQL 2: '

```
SELECT Name, Spoken_Language
FROM Countries
ORDER BY Population ASC
LIMIT 1
```

SQL 3:

```
SELECT
Name,
Official_Language,
Spoken_Language
FROM Countries
ORDER BY Population ASC
LIMIT 1
```

Example 4: "Show me the mortality trends and names of provinces with a population greater than 1,500,000."

Valid SQL Queries (3):

SQL 1:

```
SELECT n.total_deaths, p.name
```



```
FROM Number_Of_Deaths n
JOIN Province p
ON n.year = p.year
WHERE p.population > 1500000
```

```
SQL 2:
SELECT c.rate, p.name
FROM Crude_Death_Rate c
JOIN Province p
ON c.year = p.year
WHERE p.population > 1500000
```

```
SQL 3:
SELECT n.total_deaths, c.rate, p.name
FROM Number_Of_Deaths n
JOIN Province p
ON n.year = p.year
JOIN Crude_Death_Rate c
ON p.year = p.year
WHERE Province.population > 1500000
```

Example 5: "Show the birth rate, city and population for cities with a population greater than 75,000 people."

Valid SQL Queries (3):

```
SQL 1:
SELECT
b.Number_Of_Births,
b.City,
p.TotalPopulation
FROM BirthYears b
JOIN AgeGroupPopulations p
ON b.City = p.CityName
WHERE p.TotalPopulation > 75000
```

```
SQL 2:
SELECT
b.Crude_Birth_Rate,
b.City,
p.TotalPopulation
FROM BirthYears b
JOIN AgeGroupPopulations p
ON b.City = p.CityName
WHERE p.TotalPopulation > 75000
```

```
SQL 3:
SELECT
b.Number_Of_Births,
b.Crude_Birth_Rate,
b.City,
p.TotalPopulation
FROM BirthYears b
JOIN AgeGroupPopulations p
ON b.City = p.CityName
WHERE p.TotalPopulation > 75000
```

Guided by these examples, Qwen generates one query:

```
SELECT
Municipality,
Foreign_born_Population
FROM Population
WHERE Total_Population > 60000
```

D. STAGE 3: INTERPRETATION GENERATION (LORA-FINETUNED LLAMA-3.1-8B)

The GRPO-trained interpretation model analyzes the question's ambiguity and generates two interpretations:

- 1) "Show me the municipalities and foreign-born population where more than 60,000 people live."
- 2) "Show me the municipalities and naturalized citizens where more than 60,000 people live."

E. STAGE 4: SQL GENERATION FROM INTERPRETATIONS (QWEN2.5-CODER-32B)

For each interpretation, Qwen generates corresponding SQL:

```
Query for Interpretation 1:
SELECT
Municipality,
Foreign_born_Population
FROM Population
WHERE Total_Population > 60000
```

```
Query for Interpretation 2:
SELECT
Municipality,
Naturalized_Citizens
FROM Population
WHERE Total_Population > 60000
```

F. STAGE 5: DEDUPLICATION

The system identifies that Query 1 from Stage 2 is identical to the query from Interpretation 1. After deduplication, the framework retained 2 unique SQL queries.

G. STAGE 6: ERROR CORRECTION AND VALIDATION

Both queries execute successfully against the database:

H. STAGE 7: EVALUATION AGAINST GROUND TRUTH

The SQL evaluator computes standard metrics using three ground-truth queries:

```
GT1:
SELECT
Municipality,
Foreign_born_Population
FROM Population
WHERE Total_Population > 60000
```

```
GT2:
SELECT
Municipality,
Naturalized_Citizens
FROM Population
WHERE Total_Population > 60000
```

```
GT3:
SELECT
Municipality,
Foreign_born_Population,
Naturalized_Citizens
FROM Population
WHERE Total_Population > 60000
```

- Precision: 1.0 (2/2 predicted queries exactly matched gold)
- Recall: 0.67 (2/3 gold queries were matched)
- F1 Score: 0.80
- Full Coverage: No (missing GT3 which combines both columns)

APPENDIX I

COMPARISON OF DIFFERENT REWARD MODELS IN GRPO CURRICULUM TRAINING

See Table 9.

REFERENCES

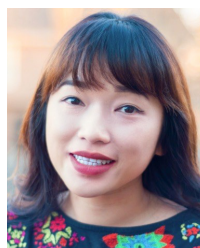
- [1] A. Jonker and T. Mucci. (2025). *What is Structured Query Language*. [Online]. Available: <https://www.ibm.com/think/topics/structured-query-language>
- [2] D. L. Waltz and B. A. Goodman, "Writing a natural language data base system," in *Proc. 5th Int. Joint Conf. Artif. Intell.*, 1977, pp. 144–150.
- [3] D. L. Waltz, "An English language question answering system for a large relational database," *Commun. ACM*, vol. 21, no. 7, pp. 526–539, Jul. 1978, doi: [10.1145/359545.359550](https://doi.org/10.1145/359545.359550).
- [4] L. R. Harris, "The ROBOT system: Natural language processing applied to data base query," in *Proc. Annu. Conf.*, 1978, pp. 165–172, doi: [10.1145/800127.804087](https://doi.org/10.1145/800127.804087).
- [5] G. G. Hendrix, E. D. Sacerdoti, D. Sagalowicz, and J. Slocum, "Developing a natural language interface to Complex data," *ACM Trans. Database Syst.*, vol. 3, no. 2, pp. 105–147, Jun. 1978, doi: [10.1145/320251.320253](https://doi.org/10.1145/320251.320253).
- [6] S. J. Kaplan, "Cooperative responses from a portable natural language data base query system," Ph.D. dissertation, Dept. Comput. Inf. Sci., Univ. Pennsylvania, Philadelphia, PA, USA, 1979. [Online]. Available: <https://www.proquest.com/dissertations-theses/cooperative-responses-portable-natural-language/docview/302969267/se-2>
- [7] S. J. Kaplan, E. Mays, and A. K. Joshi, "A technique for managing the lexicon in a natural language interface to a changing data base," in *Proc. 6th Int. Joint Conf. Artif. Intell.*, 1979, pp. 463–465.
- [8] H. L. Morgan, E. F. Codd, W. A. Martin, L. Harris, D. Sagalowicz, and I. Kameny, "Natural language interfaces," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 1978, p. 1, doi: [10.1145/509252.509254](https://doi.org/10.1145/509252.509254).
- [9] (2024). *TDWI Best Practices Report | Reducing Time to Insight and Maximizing the Benefits of Real-Time Data: Asset*. [Online]. Available: <https://tdwi.org/research/2024/03/bi-all-best-practices-report-reducing-time-to-insight-and-maximizing-the-benefits-of-real-time-data/asset.aspx?tc=assetpg>
- [10] OpenAI. (2022). *Introducing ChatGPT*. [Online]. Available: <https://openai.com/index/chatgpt/>
- [11] S. Talaei, M. Pourreza, Y. Chang, A. Mirhoseini, and A. Saberi, "CHESS: Contextual harnessing for efficient SQL synthesis," 2024, *arXiv:2405.16755*.
- [12] M. Pourreza, H. Li, R. Sun, Y. Chung, S. Talaei, G. T. Kakkar, Y. Gan, A. Saberi, F. Ozcan, and S. Arik, "CHASE-SQL: Multi-path reasoning and preference optimized candidate selection in Text-to-SQL," in *Proc. Int. Conf. Learn. Represent.*, vol. 2025, 2025, pp. 60385–60415. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/file/974ff7b5bf08dbf9400b5d59a39c77f-Paper-Conference.pdf
- [13] L. Sheng and S. Xu, "CSC-SQL: Corrective self-consistency in text-to-SQL via reinforcement learning," in *Proc. 14th Int. Joint Conf. Natural Lang. Process. 4th Conf. Asia-Pacific Chapter Assoc. Comput. Linguistics*, 2025, pp. 1473–1496. [Online]. Available: <https://aclanthology.org/2025.findings-ijcnlp.91/>
- [14] V. Shkapenyuk, D. Srivastava, T. Johnson, and P. Ghane, "Automatic metadata extraction for text-to-SQL," 2025, *arXiv:2505.19988*.
- [15] M. Pourreza, S. Talaei, R. Sun, X. Wan, H. Li, A. Mirhoseini, A. Saberi, and S. O. Arik, "Reasoning-SQL: Reinforcement learning with SQL tailored partial rewards for reasoning-enhanced text-to-SQL," 2025, *arXiv:2503.23157*.
- [16] I. Saparina and M. Lapata, "AMBROSIA: A benchmark for parsing ambiguous questions into database queries," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 90600–90628. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/file/a4c942a8405cc910f0a833d28d2573cc-Paper-Datasets_and_Benchmarks_Track.pdf
- [17] A. Bhaskar, T. Tomar, A. Sathe, and S. Sarawagi, "Benchmarking and improving text-to-SQL generation under ambiguity," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2023, pp. 7053–7074. [Online]. Available: <https://aclanthology.org/2023.emnlp-main.436>
- [18] P. J. Price, "Evaluation of spoken language systems: The ATIS domain," in *Proc. Speech Natural Lang., Workshop*, 1990, pp. 1–5. [Online]. Available: <https://aclanthology.org/H90-1020/>
- [19] D. A. Dahl, M. Bates, M. Brown, W. Fisher, K. Hunnicke-Smith, D. Pallett, C. Pao, A. Rudnick, and E. Shriberg, "Expanding the scope of the ATIS task: The ATIS-3 corpus," in *Proc. workshop Human Lang. Technol.*, Jersey, 1994, p. 43. [Online]. Available: <https://aclanthology.org/H94-1010/>
- [20] T. Yu, R. Zhang, M. Yasunaga, Y. C. Tan, X. V. Lin, S. Li, H. Er, I. Li, B. Pang, T. Chen, E. Ji, S. Dixit, D. Proctor, S. Shim, J. Kraft, V. Zhang, C. Xiong, R. Socher, and D. Radev, "SPaC: cross-domain semantic parsing in context," in *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics*, 2019, pp. 4511–4523. [Online]. Available: <https://aclanthology.org/P19-1459>
- [21] T. Yu et al., "CoSQL: A conversational text-to-SQL challenge towards cross-domain natural language interfaces to databases," in *Proc. Conf. Empirical Methods Natural Lang. Process. 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 1962–1979. [Online]. Available: <https://aclanthology.org/D19-1204/>
- [22] B. Wang, Y. Gao, Z. Li, and J.-G. Lou, "Know what I don't know: Handling ambiguous and unknown questions for text-to-SQL," in *Proc. Findings Assoc. Comput. Linguistics*, Jul. 2023, pp. 5701–5714. [Online]. Available: <https://aclanthology.org/2023.findings-acl.352/>
- [23] M. Dong, N. Ashok Kumar, Y. Hu, A. Chauhan, C.-W. Hang, S. Chang, L. Pan, W. Lan, H. Zhu, J. Jiang, P. Ng, and Z. Wang, "PRACTIQ: A practical conversational text-to-SQL dataset with ambiguous and unanswerable queries," in *Proc. Conf. Nations Americas Chapter Assoc. Comput. Linguistics, Human Lang. Technol.*, Apr. 2025, pp. 255–273. [Online]. Available: <https://aclanthology.org/2025.naacl-long.13/>
- [24] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Oct. 2018, pp. 3911–3921. [Online]. Available: <https://aclanthology.org/D18-1425/>
- [25] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, X. Zhou, M. Chenhao, G. Li, K. Chang, F. Huang, R. Cheng, and Y. Li, "Can LLM already serve as a database interface? A big bench for large-scale database grounded Text-to-SQLs," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, Curran Associates, Inc., 2023, pp. 42330–42357. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/83fc8fab1710363050bbd1d4b8cc0021-Paper-Datasets_and_Benchmarks.pdf
- [26] Z. Huang, P. K. Damalapati, and E. Wu, "Data ambiguity strikes back: How documentation improves GPT's Text-to-SQL," 2023. [Online]. Available: <https://arxiv.org/abs/2310.18742>
- [27] K. Chen, Y. Chen, N. Koudas, and X. Yu, "Reliable text-to-SQL with adaptive abstention," *Proc. ACM Manage. Data*, vol. 3, no. 1, pp. 1–30, Feb. 2025, doi: [10.1145/3709719](https://doi.org/10.1145/3709719).
- [28] I. Saparina and M. Lapata, "Disambiguate first, parse later: Generating interpretations for ambiguity resolution in semantic parsing," in *Proc. Findings Assoc. Comput. Linguistics*, Jul. 2025, pp. 16825–16839. [Online]. Available: <https://aclanthology.org/2025.findings-acl.863>
- [29] L. Qiu, J. Li, C. Su, and L. Chen, "Interactive text-to-SQL via expected information gain for disambiguation," 2025, *arXiv:2507.06467*.
- [30] R. R. Menon, K. Qian, L. Chen, I. Joshi, D. Pandyan, J. Harrison, S. Srivastava, and Y. Li, "FISQL: Enhancing Text-to-SQL Systems with Rich Interactive Feedback," in *Proc. 28th Int. Conf. Extending Database Technol. (EDBT)*, 2025, pp. 1032–1038. [Online]. Available: <https://openproceedings.org/2025/conf/edbt/paper-86.pdf>
- [31] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [32] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," 2024, *arXiv:2402.03300*.
- [33] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proc. 26th Annu. Int. Conf. Mach. Learn.*, 2009, pp. 41–48.
- [34] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021. [Online]. Available: <https://openreview.net/forum?id=nZeVKeeFYf9>
- [35] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," 2021, *arXiv:2106.09685*.

- [36] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36. Curran Associates, Dec. 2023, pp. 10088–10115. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/1feb87871436031bdc0f2beaa62a049b-Paper-Conference.pdf
- [37] Y. Bai et al., “Constitutional AI: Harmlessness from AI feedback,” 2022, *arXiv:2212.08073*.
- [38] (2018). *SPIDER: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task*. [Online]. Available: <https://yale-lily.github.io/spider>
- [39] (2023). *BIRD: Benchmark for Implicit Reasoning in Database Queries*. [Online]. Available: <https://bird-bench.github.io/>



VIJAY K. MADISETTI (Fellow, IEEE) received the Ph.D. degree in electrical engineering and computer sciences from the University of California at Berkeley. He holds the position of a Professor in cybersecurity and privacy (SCP) with Georgia Institute of Technology (Georgia Tech). He has authored several widely referenced textbooks on topics, including cloud computing, data analytics, blockchain, and microservices. He is the co-author of the IEEE Standard on VHDL RTL Synthesis Language, and the inventor of more than 80 issued U.S. patents. In addition, he has been honored with the Terman Medal by American Society of Engineering Education (ASEE) and Georgia Tech’s Outstanding Ph.D. Advisor Award.

...



MIA HOANG received the B.Sc. degree in economics from Aalto University, Finland, the M.Sc. degree in management information systems and the M.B.A. degree from the University of Nebraska Omaha, Omaha, NE, USA, and the M.Sc. degree in computer science from Georgia Institute of Technology, Atlanta, GA, USA. She is currently a researcher at the Georgia Institute of Technology and Vice President of Cybersecurity Engineering at JPMorgan Chase. She is also a Data and Analytics

Leader with more than ten years of professional experience in the financial industry. Her research interests include the development of AI and data-driven solutions to democratize analytics and personalize educational experiences for learners of all ages.